Requirements as the Oracle

Generating REST API Tests from Vague Requirements Without Learning the Bugs They Should Catch

Final report

Team PreTrained

Sohail Memon	2025201075	Mohd Ahmad	2025201029
Kathan Patel	2025201039	Md Taufique Hussain	2025202007

Code and results: `https://github.com/SohailTSM/Requirements-as-the-Oracle`

Abstract

An automatically generated API test is only useful if it checks what the API *should* do. We built an agent that writes REST API tests from natural-language requirements, using one small open model, Llama 3.1 8B. We tackle two problems. First, real requirements are often vague. Second, when a test is repaired by running it against a buggy service, the model tends to copy the buggy output into the test, so the test "learns" the bug. Our agent finds the relevant API operations, lists the checks a requirement implies, writes a structured plan instead of code, cleans the plan using the API specification, compiles it into a pytest file, runs it, and repairs it under a guard that blocks copied values. On RESTestBench (3 services, 106 requirements, 228 planted bugs) with vague requirements, it catches 12.7% of the planted bugs, against 2.6% for one-shot generation ($p < 0.001$). This is on par with RESTestBench's own repair loop (11.3%). When tests are repaired against the buggy service, which is the realistic case, our agent keeps its score (14.8%) while the repair loop falls to 4.3% (3.4×, $p < 0.001$). Ablations show that two components carry the gain: the plan-and-compile executor and the specification clean-up. Two human labellers agreed strongly with each other ($\kappa = 0.84$), but not with the Llama judge we used for offline scoring, so we report the human-labelled numbers.

1 Introduction

Most automatic REST testing tools (EvoMaster, RESTler, AutoRestTest [2]) only flag crashes (HTTP 5xx) or responses that break the schema. They cannot see a logic bug that returns `200 OK`, for example a user who is allowed to delete another user's data. To catch such bugs, a test must know the *expected* behaviour, and that is written in the requirements.

RESTestBench [1] measured how well LLMs turn requirements into tests. They do well with precise requirements but poorly with vague ones, which are the common case. A repair loop (run the test, show the model the HTTP trace, let it rewrite the test) helps. However, on a buggy service the model "adapts its assertions to match the faulty behaviour": the repaired test treats the bug as correct. The paper measures this but offers no fix. Other work also shows that LLM-written test checks tend to capture what the code *does*, not what it *should* do [9]. Our own re-analysis of RESTestBench's released files found that its repair prompt even contains an example that teaches copying the observed value. For Llama 3.1 8B on vague requirements, the whole gain from repair (3.1% → 12.3%) disappears when the repair happens on the buggy service (3.5%).

Goal and users. We want an agent that writes useful tests from a vague requirement and an OpenAPI specification alone, with a small model that a backend or QA engineer can run locally, and whose repair loop cannot learn the bug. The agent never asks clarifying questions: in the vague setting there is nobody to answer them.

Research questions. RQ1: Can an 8B agent catch bugs from vague requirements, and how much of the gap to precise requirements does it close? **RQ2:** Does restricted repair stop the test from learning the bug, without raising false alarms? **RQ3:** Which components actually help? Our hypotheses: **H1**, deriving implied checks raises the vague score well above one-shot generation; **H2**, restricted repair keeps the adaptation gap near zero; **H3**, running tests through tools sharply cuts tests that fail on the correct service.

2 Literature review

- **Crash and schema fuzzers** such as AutoRestTest [2], LlamaRestTest [3] and Schemathesis judge a response only by crashes or schema errors, so they miss logic bugs that return 200. We use Schemathesis as baseline E.
- **Agents that judge the response**, such as LogiAgent [4] and ARMeta [5], let an LLM look at the response and decide if it is right. Because the judge sees the response first, it shares the bias we target. A LogiAgent-style judge is our baseline B4.
- **Oracle generation without requirements**, such as SATORI [6] and MASTOR [7], derives the expected behaviour from the specification or the code.
- **Test generation from requirements**, such as APITestGenie [8] and RESTestBench's own tools [1] (our B1 and B2). None of them protects the repair step from learning the bug.

3 Dataset

Benchmark: RESTestBench [1]. Three real web services run in Docker: FastAPI (Python), NestJS RealWorld (TypeScript) and TodoApp (.NET). They have 63, 29 and 14 requirements, each written once vaguely and once precisely, and 141, 65 and 22 hand-planted bugs (*mutants*). A generated test scores only if it passes on the correct service; it then "kills" every mutant on which it fails. We use the benchmark's scoring code unchanged. The agent sees only the requirement and the specification. We developed on FastAPI only; NestJS and TodoApp were kept out of every design decision.

Example (FastAPI, requirement 1). Vague: "Creating a valid item yields an appropriate response." Precise: "Register a new user, ... create a new item ... verify that the status code is 200, the returned item has the same title and description, the `id` field is present, and `owner_id` matches the user." Its mutants include returning 400 instead of 200 and silently replacing the description.

Data we created. The benchmark has no labels below the test level, so we built:

- **Gold operations** for all 106 requirements: which API operations each reference test calls (parsed from the benchmark's reference tests). Used to score retrieval.
- **Gold checks:** 268 small expected-behaviour checks, made by splitting the human-written precise requirements with an AI assistant and then checked by humans. Used to score the check-derivation step and as fine-tuning targets.
- **Human labels:** 181 items over 45 requirements, labelled independently by two team members.
- **Fine-tuning data:** 86 / 154 / 184 SFT examples (one fold per held-out service), and DPO preference pairs from our own repair logs, where the guarded plan is preferred over the plan that copied an observed value (1,273 from FastAPI runs; 143 for the fold we trained).

4 Methodology

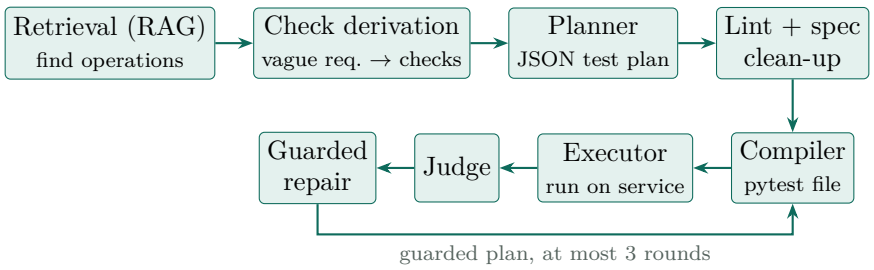


Figure 1: The agent. The top row uses only the requirement and the specification, so it cannot learn a bug. Only the bottom row touches the running service.

(1) **Retrieval (RAG).** Each API operation in the OpenAPI file is one document. Keyword search (BM25) and embedding search (bge-small) are combined, and a small re-ranking model (MiniLM)

re-orders the results for short requirements (at most 30 words). We keep the top 5 operations and add any operation they depend on, such as the POST that creates an item before it can be deleted.

(2) **Check derivation.** The model turns the requirement into small checks, guided by a list of properties requirements usually imply (round trip, echo, ownership, before/after changes). It writes 5 drafts, and we keep the checks that most drafts agree on (self-consistency). Each check is marked *meaning* (stated by the requirement, locked) or *format* (a detail it does not state, such as the exact status code).

(3) **Plan instead of code.** The model writes a JSON plan of users, API calls and checks, not Python code. A linter compares the plan with the specification (unknown operations, fields that do not exist, impossible comparisons), and the model fixes the problems once.

A shortened plan for the example requirement in Section 3 (`{{s1.req.title}}` means "the title sent in step s1"):

```
{"actors": {"A": {"type": "new_user"}},
 "steps":  [{"id": "s1", "actor": "A", "call": "POST /api/v1/items/",
             "body": {"title": "item {{rand.str}}", "description": "desc"}}],
 "checks": [{"id": "C1", "step": "s1", "expect": "status", "value": 200, "kind": "format"},
            {"id": "C2", "step": "s1", "expect": "field_equals", "field": "title",
             "value": "{{s1.req.title}}", "kind": "meaning"},
            {"id": "C3", "step": "s1", "expect": "field_equals", "field": "owner_id",
             "value": "{{A.id}}", "kind": "meaning"}]}
```

(4) **Specification clean-up.** Fixed rules, based only on the specification, then remove checks that can never pass, turn status codes the requirement does not mention into a class (`200`→`2xx`), replace made-up credentials, fix wrong references and fill in path parameters.

(5) **Compiler and executor.** A compiler turns the plan into a self-contained pytest file. Its small runtime creates users with valid credentials, logs them in, passes IDs between steps and sends every call to the service under test. Authentication is read from the specification, not hard-coded.

(6) **Guarded repair and judge.** If the test fails, a judge decides who is wrong. It may blame the test only by quoting the requirement or the specification; otherwise it blames the API and the loop stops. If the test is to be repaired:

- the repair model sees status codes and field names, but never the response *values*, so it cannot copy them;
- a guard puts back any changed or removed meaning check, and blocks any new expected value that equals a value the service just returned;
- a field may be re-pointed only to a field the specification documents.

(7) **Fine-tuning.** We tried QLoRA fine-tuning (SFT) of the check-derivation step and DPO that prefers "report the bug" over "copy the value", trained with Unsloth and TRL on Kaggle T4 GPUs.

We also built and compared other designs: a single ReAct agent that calls tools, two multi-agent designs, tools served over MCP, two agents talking over A2A, and three amounts of information passed from the judge to the repair step.

5 Experiment runs

Model and compute. Every LLM role, in our agent and in all baselines, uses Llama 3.1 8B Instruct in 4-bit form (Q4_K_M) under Ollama, with a 24k-token context, temperature 0.2 and at most 3 repair rounds. The model ran on free Kaggle T4 GPUs from the four team accounts, reached through Cloudflare tunnels. The services ran in Docker on a laptop; we cloned them on different ports so that up to 10 copies of a service could run at once. A local dispatcher that we wrote balanced all LLM requests across the GPUs and retried failed requests on another server: 105,044 requests, 99.8% answered, 251M tokens, no paid APIs.

How the runs were executed. A queue runner started each job only if it could finish before the GPU sessions ended, split long runs into parts of at most 1.5 hours, and paused when the laptop ran low on memory. Before trusting any score, we ran the benchmark's own 106 hand-written reference tests on every service and every copy: all passed on the correct service and caught 100% of the planted bugs, so the environment was sound. All baselines were re-run on the same 4-bit model; our one-shot baseline matched the published number (2.2% vs 2.0% on vague requirements).

Baselines. B1 RESTestBench one-shot generation; **B2** RESTestBench repair loop, unchanged; **B3** B2 with only its prompts fixed (copying example removed, judge allowed to say "API bug"); **B4** a LogiAgent-style response judge; **E** the Schemathesis fuzzer, which reads no requirements.

Metrics. *Caught*: RESTestBench's score, the share of planted bugs killed by tests made while working with the correct service. *Mutant-gen*: the same score for tests made while working with the *buggy* service, the realistic case, since in practice you do not know if your code is correct. *Adaptation gap*: Caught minus Mutant-gen; a large gap means the tool learns bugs. *Fail-OK*: the share of tests that fail on the correct service (false alarms). We also measure each step on its own (Table 4).

Protocol.

- **Sweep:** change one component at a time on FastAPI, vague, 1 run each (29 variants). The default ran 3 times to measure noise.
- **Finals:** the final configuration and 2 runner-ups, on all services and both wordings, 3 runs each. Every baseline also ran 3 times.
- **Ablations:** the final configuration with one component removed, all services, vague, 1 run each.
- **Statistics:** the unit is the requirement ($n = 106$), its score averaged over runs. We use the paired Wilcoxon signed-rank test with Holm correction for multiple comparisons, rank-biserial r as the effect size, and bootstrap 95% confidence intervals (5,000 resamples of requirements).
- **Freeze:** every decision was logged with its reason, and experiments were frozen on 11 Nov.

6 Evaluation

6.1 RQ1: vague requirements

Tool	Wording	FastAPI	NestJS	TodoApp	**All**	Mutant-gen	Fail-OK
B1 one-shot	vague	4.3	0.0	0.0	2.6 ± 0.9	–	97.2
B2 repair loop	vague	**17.5**	0.0	4.5	11.3 ± 1.4	4.3 ± 2.0	85.2
B3 prompt-fixed	vague	4.5	0.0	0.0	2.8 ± 1.8	3.1 ± 0.9	96.9
B4 response judge	vague	6.6	3.1	18.2	6.7 ± 1.1	–	**40.6**
E fuzzer	–	13.9	0.0	9.1	9.5 ± 4.8	–	70.8
Ours (final)	vague	12.5	2.6	**43.9**	**12.7** ± 2.7	**14.8** ± 1.2	70.4
B1 one-shot	precise	39.7	5.1	1.5	26.2 ± 1.8	–	77.4
B2 repair loop	precise	**47.8**	6.2	0.0	**31.3** ± 0.7	**24.8** ± 0.6	68.6
B3 prompt-fixed	precise	37.6	5.6	0.0	24.9 ± 0.3	25.4 ± 3.0	78.0
B4 response judge	precise	13.5	10.3	4.5	11.7 ± 2.9	–	**56.0**
Ours (final)	precise	20.6	**22.1**	1.5	19.2 ± 2.2	16.0 ± 3.0	77.7

Table 1: Planted bugs caught (%), mean over 3 runs ($\pm$ sd for the total). "–": the tool does not repair against the service, so it has no mutant-gen score.

Ours (final) vs	Difference	95% CI	p (Holm)	r
B1, vague, caught	+10.1	[+5.2, +15.2]	<0.001	+0.75
B3, vague, caught	+9.9	[+5.5, +14.6]	<0.001	+0.85
B4, vague, caught	+6.0	[+1.0, +11.1]	0.032	+0.45
B2, vague, caught	+1.5	[−5.3, +7.9]	1.000	+0.09
B2, vague, **mutant-gen**	**+10.5**	[+5.6, +15.7]	<0.001	+0.69
B2, precise, caught	−12.1	[−20.8, −3.5]	0.062	−0.49

Table 2: Paired comparisons over the 106 requirements, in percentage points (ours minus the other tool). The full set is in `results/tables/stats.md`.

- **Much better than one-shot generation:** +10.1 points over B1 and +9.9 over B3 ($p < 0.001$, large effects). This closes 35% of the gap between B1 on vague requirements (2.6) and the best precise score (B2, 31.3).
- **A draw with B2 on the correct service** (+1.5, not significant).
- **A clear win in the realistic setting:** when tests are repaired against the buggy service, we keep 14.8% while B2 falls to 4.3% (+10.5 points, 3.4×).
- **Held-out services:** B2 is strongest on FastAPI, but we win on the services we never tuned on (TodoApp vague 43.9 vs at most 18.2; NestJS precise 22.1 vs at most 10.3). On TodoApp the baselines register users with weak passwords like `password123`, which an undocumented password rule rejects; our runtime creates valid users.
- **Weakness: precise wording.** B2 beats us by 12.1 points. Our clean-up turns exact status codes into classes and drops specific checks, which throws away the detail a precise requirement gives. Our score rises only from 12.7 to 19.2, while B2's nearly triples.

6.2 RQ2: does the test learn the bug?

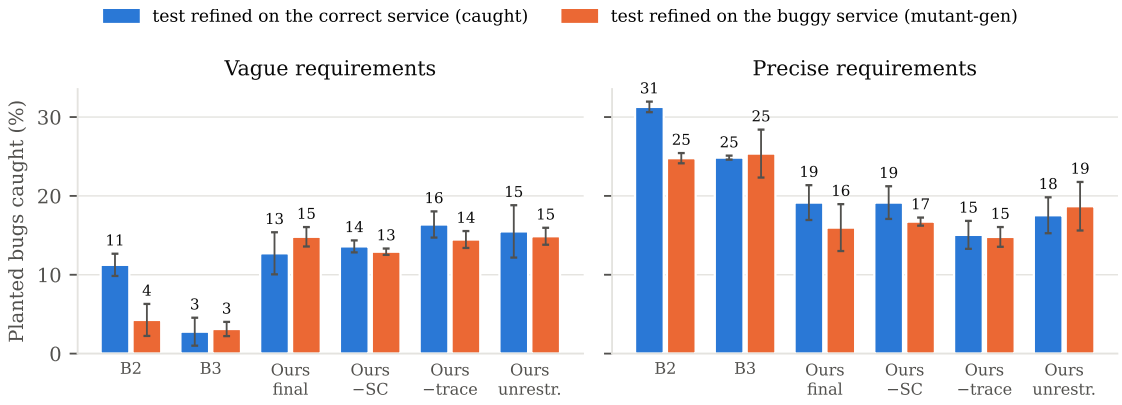


Figure 2: Bugs caught by tests repaired on the correct service (blue) and on the buggy service (orange), mean ± sd over 3 runs. B2 loses most of its score on the buggy service; our variants do not. SC = self-consistency.

Tool	Wording	Gap	p	Repair rounds copying a value	Copies reaching the test
B2 repair loop	vague	**7.0 ± 2.8**	0.004	261 / 1931 (13.5%)	all
B2 repair loop	precise	**6.5 ± 1.3**	0.004	151 / 1825 (8.3%)	all
Ours (final)	vague	−2.1 ± 3.4	0.058	21 / 739 tried (2.8%)	3 / 787 (0.4%)
Ours (final)	precise	3.2 ± 3.0	0.30	57 / 960 tried (5.9%)	8 / 972 (0.8%)
Ours, unrestricted	vague	0.6 ± 3.8	0.68	–	33 / 807 (4.1%)
Ours, unrestricted	precise	−1.1 ± 2.1	0.20	–	135 / 932 (14.5%)

Table 3: Adaptation gap (caught minus mutant-gen) and value copying during repair. For our agent, "copies reaching the test" compares each next plan that runs with the values the service returned; the few left are format changes the specification allows, or values that equal the correct behaviour.

- **B2 learns the bugs.** Its adaptation gap is 7.0 points on vague and 6.5 on precise wording (both $p = 0.004$). In 8–14% of its repair rounds it adds an expected value copied from the previous response. B3, whose judge almost always says "API bug", hardly repairs at all, so it neither gains nor adapts.
- **Our agent does not, in any variant.** No gap differs from zero (final: −2.1 vague, +3.2 precise, both n.s.).
- **The guard works,** but it is not the main reason. It blocked 512 (vague) and 773 (precise) tampering attempts and cut copied values reaching the test from 4.1–14.5% to under 1%. Yet our *unrestricted* variant, which copies values freely, also shows no gap and no more false alarms (Fail-OK 68.2 vs 70.4).
- **Why:** the structure protects the test. Checks are fixed from the requirement before anything runs, most repair rounds (58–70%) fix setup errors rather than expectations, and a copied value in our plans usually equals correct behaviour that the planted bug does not touch.

6.3 Each step on its own

Step	Metric	Final configuration
Retrieval (RAG)	Recall@5 / MRR	vague 0.89 / 0.87; precise 0.90 / 0.78 (BM25 alone: 0.79 / 0.70)
Check derivation	Gold checks fully found / partly found; checks contradicting the gold (human labels)	0.29–0.41 / 0.54–0.58; contradictions 0.09
Planner	Plans that fail to parse; plans with lint issues	2.9% / 51% (vague)
Executor (tool calls)	Repair rounds with a setup error	58% vague, 70% precise
Judge	"API bug" verdicts on the correct service, where every failure is the test's fault	212 of 213: it does not tell the cases apart
Safety	Tampering attempts blocked / accepted	512 / 0 vague, 773 / 0 precise
Cost	LLM calls / tokens / time per test	7.7 / 13.9k / 123 s (vague)

Table 4: Step-level metrics from the agent logs of the final configuration (3 runs, 1,002 tests per wording).

The 8B judge almost never blames the test, so in practice it works as a stop rule. RESTestBench's own judge behaves the same way in B3. A judge that must quote evidence seems to need a stronger model.

6.4 Human validation

Two team members each labelled the same 181 items (45 requirements): 63 judgements of whether a gold check is correct, and 118 judgements of how well our derived checks cover the gold checks. The

humans agreed strongly with each other (Cohen's $\kappa = 0.84$), and 125 of 126 gold-check judgements were "correct", so the gold data is sound. The Llama judge we had used for offline scoring did not agree with them ($\kappa = 0.03$ for coverage, 0.27 for derived checks): it reported three times as many contradictions and often called "partial" coverage "full". So we report only the human-labelled numbers for this step. The main results do not use this judge, because the bug scores come from the benchmark.

6.5 Failure analysis

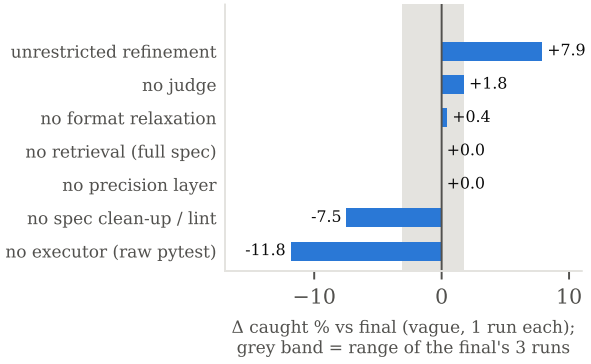
What happened to the test made on the correct service (3 runs × 106)	Vague	Precise
Passes and catches at least one bug	63 (20%)	61 (19%)
Passes but catches no bug (checks too weak)	31 (10%)	10 (3%)
Fails: setup error (wrong field, rejected setup call)	85 (27%)	136 (43%)
Fails: a check is wrong (judge blamed the API, or rounds ran out)	124 (39%)	96 (30%)
No valid plan	15 (5%)	15 (5%)

Table 5: Where tests go wrong, from the agent logs and the benchmark results.

Wrong checks and setup errors are the two main problems. The most common setup errors are FastAPI calls that need admin rights (403) and references to request fields that do not exist. Three examples:

- **Checks that contradict each other.** Self-consistency once merged "deleting another user's item is rejected (4xx)" with "... still returns 200". The test can never pass. A consistency check over the derived checks would remove this kind of error.
- **A misleading specification.** A check expected status 422 for a wrong password because the specification documents 422, but the service returns 400. The specification is not always right.
- **The guard at work.** On one mutant, the repair step tried to add a status check equal to the buggy status it had just seen. The guard blocked it, and the locked check kept the test able to catch the bug.

7 Ablations (RQ3: which components matter)



Removed (vague, 1 run)	Caught	Fail-OK
nothing (final, 3 runs)	12.7	70.4
executor (raw pytest)	0.9	98.1
spec clean-up / lint	5.3	85.8
check derivation	12.7	75.5
retrieval (full spec)	12.7	71.7
format relaxation	13.2	74.5
judge	14.5	65.1
restricted repair	20.6	60.4

Figure 3: The final configuration with one component removed, all services, vague. The grey band is the range of the final's own 3 runs (9.6–14.5); anything inside it is noise.

- **Two components carry the result.** Without the plan-and-compile executor, 98% of tests fail on the correct service and the score falls to 0.9%; the model writes placeholders such as `Bearer YOUR_TOKEN`. Without the clean-up and lint, the score falls to 5.3%. The clean-up is busy: per 1,002 tests it removed 443 impossible checks, replaced 253 made-up credentials and generalised 252 status codes.

- **No clear effect:** check derivation, retrieval, format relaxation and the judge all stay inside the noise band. Without the judge, caught rises slightly but mutant-gen falls (14.8 → 10.4), which fits the judge acting as an early stop on buggy services.
- **Removing the repair restriction** gave +7.9 in one run, but three fresh runs gave 15.5 ± 3.3 ($p = 0.27$): it did not replicate.
- **The sweep winners did not add up.** The final combined the two best sweep variants (full trace passed to repair, self-consistency), but in the finals each runner-up (the final minus one of them) scored as well as the final. Single runs vary by about ±6 points, so the sweep picked up noise.
- **Fine-tuning:** the fine-tuned check step scored 17.0, SFT for the whole agent 15.6 and SFT+DPO 14.2, against 11.3–13.5 for the default (single runs, inside the noise). We therefore trained only one DPO fold.
- **Other designs:** a ReAct agent scored 5.0 and a blackboard multi-agent design 3.5, against 12.0 for the fixed pipeline. Only 66% of ReAct's turns were well-formed tool calls, so the 8B model is not a reliable controller. MCP (12.1) and A2A (14.9) made no difference. An fp16 run (13.6) showed no loss from 4-bit weights.

8 Discussion of the hypotheses

H1, partly supported: the agent clearly beats one-shot generation, but the gain comes from executable structure, not from deriving implied checks (removing that step changes nothing). **H2, supported for the pipeline:** the adaptation gap is near zero for all our variants and 7.0 for B2, but it is the plan structure, more than the guard, that protects the test. **H3, supported:** the executor is the largest single effect, yet 70% of tests still fail on the correct service, so false alarms are reduced, not solved.

9 Limitations

- **Small scale:** one benchmark with 3 services; with 106 requirements and 3 runs, differences below about 5 points cannot be detected.
- **Noise:** the final's own runs range from 9.6 to 14.5, and sweep and ablation rows are single runs.
- **One development service:** all design choices were made on FastAPI, and some clean-up rules reflect lessons learned there.
- **One model:** all results are for Llama 3.1 8B in 4-bit; findings about the judge and ReAct may not hold for larger models.
- **Fine-tuning** was tested only in single runs, with one DPO fold.
- **Changes from the proposal:** reported runs use 4-bit weights (one fp16 run showed no difference); the GPT-5 Nano reference was dropped to keep the study to one open model; DPO was trained on one fold only.

10 Conclusion

With a small 8B model, structure matters more than extra reasoning. A plan-compile-execute pipeline with specification clean-up lifts vague-requirement testing from 2.6% to 12.7% of planted bugs, and it stays robust when tests are repaired against a buggy service, where RESTestBench's repair loop loses two-thirds of its score (14.8% vs 4.3%). The parts that asked the model to reason more (derived checks, the evidence judge, a ReAct controller, fine-tuning) did not measurably help. Future work: a consistency check over derived checks, a stronger and validated judge, keeping the details of precise requirements, and more services and runs.

Reproducibility. Code, configurations, result files and the decision log are at `https://github.com/SohailTSM/Requirements-as-the-Oracle`. The README explains the setup and how to run

one job or a whole queue, and every number in this report is regenerated from the result files by `make_tables.py`, `final_stats.py` and `make_figures.py`.

Acknowledgment

We thank our course instructor, Prof. Vasudeva Varma, and the TAs for their guidance, the RESTestBench authors for releasing their benchmark, and Kaggle for free GPU time.

References

[1] L. Kogler, S. Hangler, M. Ehrhart, B. Dornauer, R. Wuersching, P. Schrammel. *RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements.* EASE 2026. arXiv:2604.25862.

[2] M. Kim et al. *A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs (AutoRestTest).* ICSE 2025. arXiv:2411.07098.

[3] M. Kim et al. *LlamaRestTest: Effective REST API Testing with Small Language Models.* FSE 2025. arXiv:2501.08598.

[4] *LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents.* arXiv:2503.15079, 2025.

[5] *Multi-Agent LLM-based Metamorphic Testing for REST APIs (ARMeta).* arXiv:2605.28321, 2026.

[6] J. C. Alonso et al. *SATORI: Static Test Oracle Generation for REST APIs.* ASE 2025. arXiv:2508.16318.

[7] *MASTOR: A Multi-Agent Approach to Semantic Test Oracle Generation for RESTful APIs.* arXiv:2606.10465, 2026.

[8] *APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs.* arXiv:2604.02039, 2026.

[9] M. Konstantinou et al. *Do LLMs Generate Test Oracles that Capture the Actual or the Expected Program Behaviour?* arXiv:2410.21136, 2024.